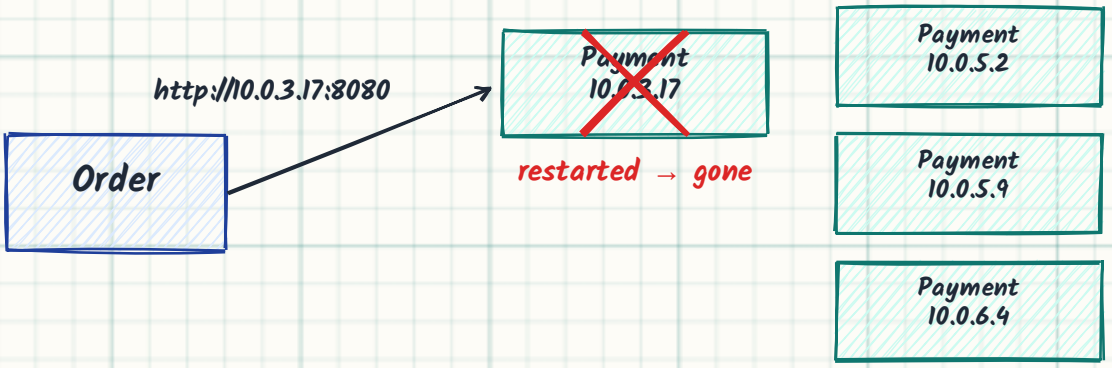
Service Discovery

Lesson 9 · how services find each other, and load balancing

In this lesson → ① Hard-coded addresses ② Service registry ③ Client-side discovery ④ Server-side discovery ⑤ Kubernetes ⑥ Load balancing ⑦ Health checks ⑧ Spring Cloud code ⑨ Interview Q&A

Running project: ShopNest, our e-commerce app (users, products, orders, payments, notifications).

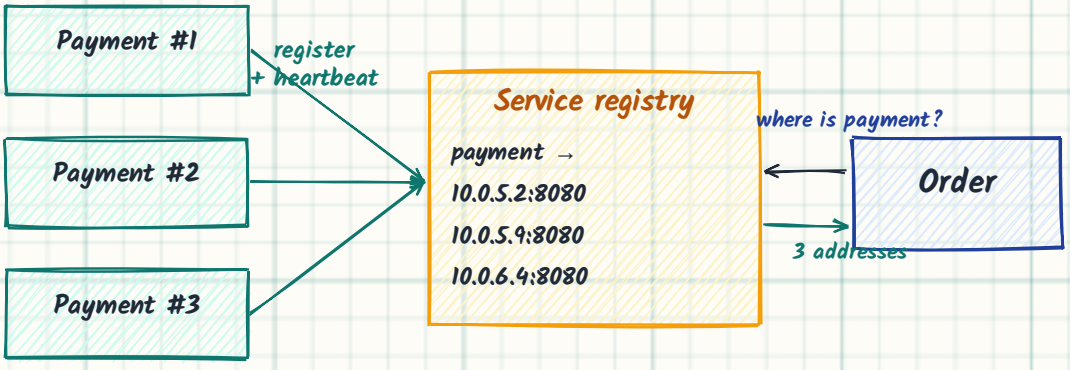
① The problem: hard-coded addresses



- ✗ Instances **restart** with new IPs
- ✗ **Autoscaling** adds and removes instances all the time
- ✗ Hard-coded address → **failed calls**, no load balancing

② The service registry

> **Service registry** = a live phone book of services: service name → list of healthy instance addresses.

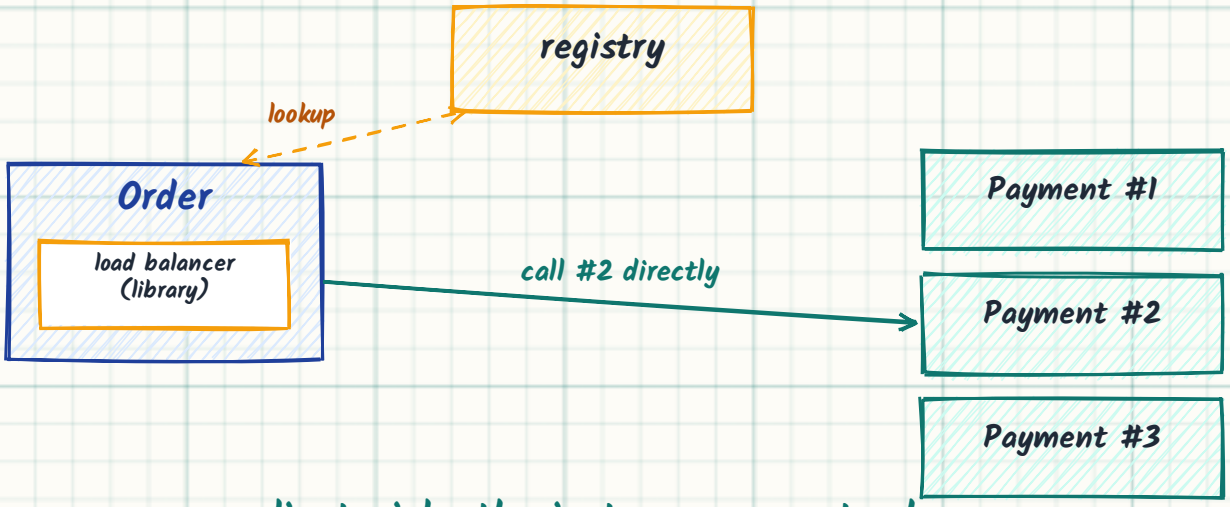


1. Instance starts → **registers** (name + address)
2. Sends **heartbeats** while alive
3. Missed heartbeats → **removed**
4. Callers **look up by name** → get current addresses

Registration style	Who registers?	Example
Self-registration	the service itself	Eureka client in Spring Boot
Third-party	the platform (registrar)	Kubernetes control plane

Registries: **Netflix Eureka**, **HashiCorp Consul**, **ZooKeeper**, **etcd** (used by Kubernetes), and DNS.

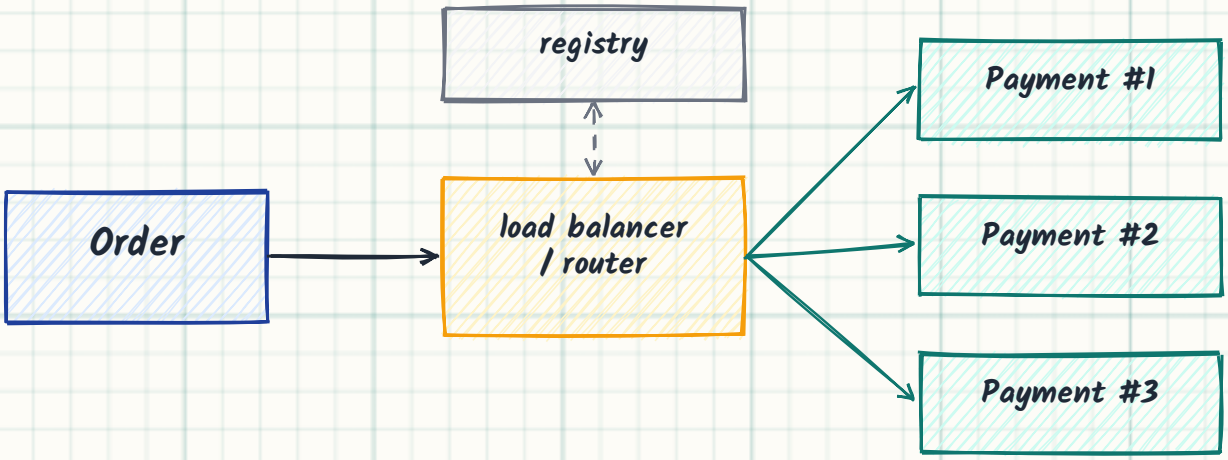
③ Client-side discovery



client picks the instance · no extra hop

✓ No extra hop
✓ Smart balancing per client
✗ A library in every client / language
✗ Clients coupled to the registry

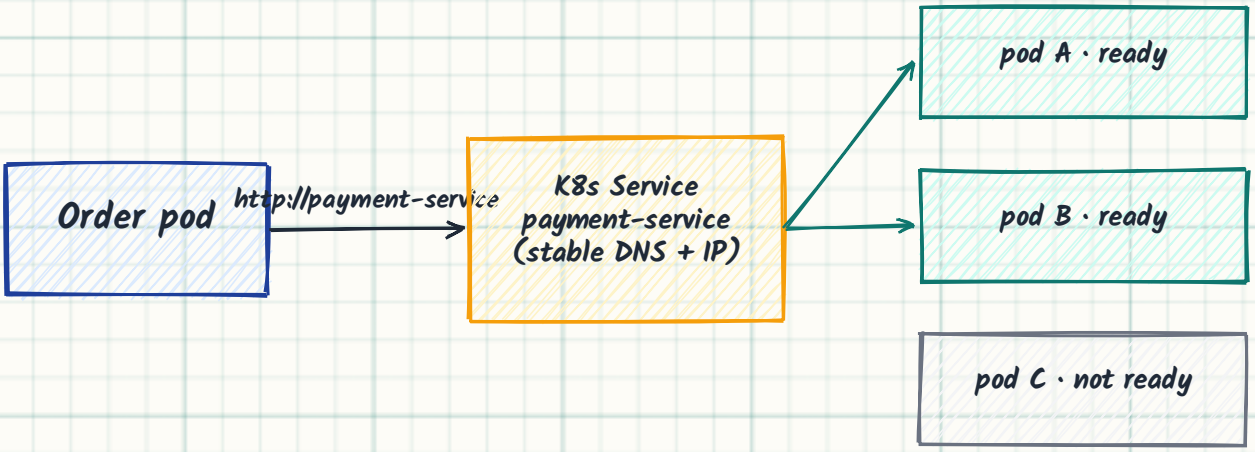
④ Server-side discovery



simple client · one extra hop

- ✓ Simple clients, any language
- ✓ Central control of routing
- ✗ One extra network hop
- ✗ Load balancer must be highly available

⑤ Discovery in Kubernetes

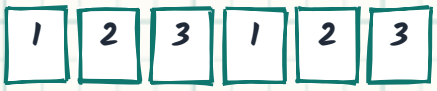


Kubernetes does discovery for you

- → A **Service** gets a stable DNS name (payment-service) and virtual IP
- → Traffic goes only to **ready** pods (readiness probe)
- → On Kubernetes you usually **don't need Eureka** (Lesson 20)

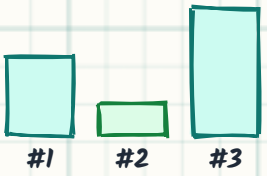
⑥ Load-balancing strategies

Round robin



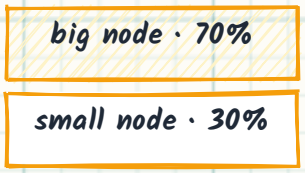
take turns

Least connections



→ #2 (fewest active)

Weighted



Strategy	How it picks	Good when
Round robin	take turns	requests are similar
Random	pick any	simple, many clients
Least connections	fewest active requests	request times vary
Weighted	share by capacity	machines differ in size

⑦ Health checks

- ✓ **Liveness**: is the process alive? (restart if not)
- ✓ **Readiness**: can it take traffic now? (remove from rotation if not)
- ✓ Spring Boot Actuator: /actuator/health, /actuator/health/readiness

Analogy
Phone contacts: you call **"Mom"**, not a number. If her number changes, the contact updates. That's discovery.

⑧ In code: Spring Cloud + Eureka

```
# order-service application.yml (with Eureka)
eureka.client.service-url.defaultZone: http://registry:8761/eureka

@Bean @LoadBalanced              // client-side load balancing
RestClient.Builder restClientBuilder() { return RestClient.builder(); }

// "payment-service" is a NAME, resolved via the registry
restClientBuilder.build().post()
   .uri("http://payment-service/payments").body(req).retrieve();
```

→ @LoadBalanced → *names are resolved via the registry and balanced on the client*

→ *Call* http://payment-service/...: *a* **name**, *not an address*

→ *The gateway uses the same idea:* uri: lb://order-service *(Lesson 8)*

⑨ Choosing an approach

Situation	Use
Running on Kubernetes	built-in Services (DNS) + readiness probes
VMs / no orchestrator, Spring stack	Eureka or Consul + client-side LB
Cloud with managed load balancers	server-side discovery (cloud LB)
Need mTLS, retries between services	add a service mesh (Istio, Linkerd)

⑩ Interview Corner

Q. What is service discovery?

A. A mechanism for services to find the current network locations of other services via a registry, instead of hard-coded addresses.

Q. Client-side vs server-side discovery?

A. Client-side: the caller queries the registry and picks an instance. Server-side: a load balancer/router does it for the caller.

Q. How does Kubernetes do service discovery?

A. A Service provides a stable DNS name and virtual IP that routes only to ready pods.

Q. Why heartbeats?

A. To detect and remove instances that crashed without deregistering.

Q. Name two load-balancing strategies.

A. Round robin (take turns) and least connections (fewest active requests).

⚡ Revise in 60 seconds

- → Never **hard-code** addresses
- → **Registry**: register + heartbeat
- → Call by **name**
- → Eureka · Consul · etcd · DNS
- → **Client-side**: no hop, needs library
- → **Server-side**: simple client, +1 hop
- → **K8s Service** = built-in discovery
- → Health: liveness vs readiness

Next lesson → MS-10
Database per service & data ownership.
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**